

Rapport — Mise en place et évaluation du système RAG

Sommaire :

 Introduction.....	3
1.1. Contexte du projet.....	3
1.2. Objectif du projet.....	3
1.3. Objectifs techniques et scientifiques.....	4
1.4. Problématique et enjeux.....	4
1.5. Structure du rapport.....	5
 2. Architecture du système.....	5
2.1. Vue d'ensemble.....	5
2.2. Schéma fonctionnel général.....	6
2.3. Les composants du système.....	7
♦ a) Le moteur linguistique : Mistral.....	7
♦ b) Le moteur de recherche sémantique : FAISS.....	7
♦ c) La base de données SQL : PostgreSQL.....	7
♦ d) Le SQL Tool.....	8
♦ e) Le Plot Tool.....	8
♦ f) L'interface utilisateur Streamlit.....	9
 3. Construction des composants.....	9
3.1. Préparation et structuration des données.....	9
♦ Données textuelles — Corpus Reddit.....	9
♦ Données tabulaires — Fichier Excel NBA.....	10
3.2. Création de la base de données PostgreSQL.....	10
3.3. Construction du moteur FAISS.....	11
Étapes clés :.....	11
3.4. Développement du SQL Tool — NBAQueryTool.....	11
 Exemple de fonctionnement :.....	11
3.5. Intégration du Plot Tool.....	12
 Exemple d'utilisation :.....	12
3.6. Intégration globale dans Streamlit.....	13
 4. Évaluation expérimentale et résultats comparatifs.....	13
4.1. Objectif de l'évaluation.....	13
4.2. Méthodologie.....	14
♦ a) Outil d'évaluation — RAGAS.....	14
♦ b) Jeu de test — eval_data.json.....	14
♦ c) Deux pipelines comparés.....	15

4.3. Résultats comparatifs (RAGAS).....	15
4.4. Analyse des résultats.....	16
♦ a) Faithfulness (forte amélioration).....	16
♦ b) Answer Similarity : légère baisse.....	16
♦ c) Context Precision.....	17
♦ d) Context Recall : légère hausse.....	17
4.5. Interprétation globale.....	17
4.6. Interprétation des résultats en lien avec les problématiques métier.....	18
♦ 1. Fiabilité analytique et vérifiabilité des données.....	18
♦ 2. Intégration équilibrée entre données objectives et analyses qualitatives.....	18
♦ 3. Gains métiers : rapidité, cohérence et explicabilité.....	18
♦ 4. Limites et perspectives d'évolution.....	19
🌱 Conclusion métier.....	19
🌱 5. Tests de robustesse sur scénarios mixtes (SQL + texte).....	20
5.1. Objectif des tests mixtes.....	20
5.2. Jeu de test exécuté.....	20
5.3. Analyse des résultats.....	21
✅ Réussite : cas purement textuel.....	21
⚠ Échecs : cas mixtes SQL + texte.....	21
5.4. Interprétation des résultats.....	22
5.5. Axes d'amélioration.....	22
5.6. Synthèse critique.....	23
⚖ 6. Analyse critique : biais, limites et pistes d'amélioration.....	23
6.1. Biais structurels et sources d'erreurs.....	23
♦ a) Biais de mapping NL → SQL.....	23
♦ b) Biais de données (corpus et base SQL).....	24
♦ c) Biais de pondération entre texte et chiffres.....	25
6.2. Limites techniques observées.....	25
♦ a) Couverture des cas d'usage.....	25
♦ b) Dépendance au modèle Mistral.....	25
♦ c) Interprétation contextuelle limitée.....	26
6.3. Pistes d'amélioration.....	26
🧠 ① Amélioration du mapping NL → SQL.....	26
🧱 ② Enrichissement de la base SQL.....	26
🗣 ③ Amélioration du moteur RAG textuel.....	26
📊 ④ Optimisation du module PlotTool.....	27
🌱 ⑤ Renforcement de la fusion SQL + FAISS.....	27
6.4. Conclusion critique.....	28
🚀 7. Synthèse finale et perspectives d'évolution.....	28
7.1. Bilan global du projet.....	28
♦ a) Évaluation quantitative (RAGAS).....	29
♦ b) Évaluation qualitative.....	29

♦ c) Visualisation et interprétation.....	30
7.3. Apports du projet.....	30
7.4. Limites identifiées.....	30
7.5. Perspectives d'évolution.....	31
7.6. Conclusion générale.....	33

Introduction

1.1. Contexte du projet

Ce projet s'inscrit dans une démarche d'exploration et de mise en œuvre d'un système de **question-réponse intelligent** appliqué à l'univers de la NBA.

L'objectif initial était de concevoir un **assistant conversationnel capable de fournir des réponses précises et contextualisées** à partir d'un ensemble de documents textuels et statistiques, en combinant la **compréhension linguistique du langage naturel (NLP)** avec des **sources de données structurées**.

Au démarrage, le système reposait sur deux principales sources d'information :

- des **fichiers PDF** issus de discussions Reddit, représentant le corpus textuel,
- un **fichier Excel** contenant des statistiques globales sur les joueurs et les équipes NBA (pourcentages de réussite, efficacité offensive, net rating, etc.).

Ces données ont servi à construire une première version d'un **pipeline RAG (Retrieval-Augmented Generation)** :

le modèle de langage (LLM) — ici **Mistral** — générait des réponses fondées sur le **contexte textuel extrait via FAISS**.

Cependant, cette première approche présentait une limite majeure : les réponses restaient qualitatives et approximatives, ne permettant aucune comparaison. Le chatbot ne pouvait répondre qu'à des questions simples non mixtes.

1.2. Objectif du projet

Face à cette contrainte, le projet a évolué vers la mise en place d'un **système RAG enrichi**, intégrant des capacités d'interrogation numérique et de raisonnement quantitatif.

L'objectif était double :

1. **Renforcer la fiabilité des réponses** en permettant à l'assistant d'effectuer de vraies requêtes sur des données chiffrées ;
 2. **Améliorer la robustesse et la pertinence** des réponses sur des questions mêlant texte et données numériques.
- 34

Pour atteindre cet objectif, une **base de données PostgreSQL** a été créée à partir du fichier Excel NBA, organisée autour de trois tables principales :

- **players** : informations générales sur les joueurs (identité, équipe, âge, etc.),
- **stats** : statistiques globales (points, rebonds, passes, pourcentages de réussite, ratings, etc.),
- **teams** : bilans et performances des équipes (seulement les noms des équipes).

L'introduction de cette base SQL a permis d'ajouter une **nouvelle dimension au système RAG**, en combinant deux types de raisonnement :

- **contextuel (textuel)**, via la recherche sémantique FAISS dans les documents PDF ;
- **quantitatif**, via un outil de génération et d'exécution automatique de requêtes SQL.

1.3. Objectifs techniques et scientifiques

Les travaux ont ainsi consisté à :

- concevoir et évaluer un pipeline **RAG complet**, reposant sur le modèle **Mistral** pour la génération,
- mettre en place un **SQL Tool** pour interpréter automatiquement les requêtes chiffrées formulées en langage naturel,
- intégrer un **PlotTool** capable de générer des graphiques à la volée à partir des résultats SQL (facultatif),
- combiner ces briques dans un **agent multimodal robuste** (texte + chiffres + visuel) au sein d'une interface interactive **Streamlit**,
- et enfin, **évaluer quantitativement** les performances du système avant et après intégration du SQL Tool, grâce au framework **RAGAS** (métriques : faithfulness, answer similarity, context precision, context recall).

1.4. Problématique et enjeux

La problématique principale de ce projet repose sur la question suivante :

- **Comment améliorer la pertinence et la robustesse d'un système RAG en y intégrant une composante de raisonnement numérique ?**

L'enjeu est d'évaluer dans quelle mesure l'ajout d'un module SQL permet :

- d'augmenter la **fidélité** et la **vérifiabilité** des réponses,
- sans pour autant dégrader leur **cohérence contextuelle** ou leur **fluidité linguistique**.

Au-delà de la performance pure, ce projet interroge la **capacité des modèles de langage à interagir avec des bases de données structurées**, tout en maintenant une compréhension sémantique riche des textes.

1.5. Structure du rapport

Le présent rapport s'articule autour des étapes suivantes :

1. La **présentation de l'architecture** du système et de ses composants ;
2. La **mise en œuvre technique** du pipeline RAG enrichi ;
3. L'**évaluation expérimentale** avant et après intégration du SQL Tool ;
4. Les **tests de robustesse** sur des scénarios mixtes (texte + données chiffrées) ;
5. Une **analyse critique** des biais, limites et pistes d'amélioration ;
6. Et enfin, la **conclusion**, mettant en perspective les apports de ce travail et ses prolongements possibles.

2. Architecture du système

2.1. Vue d'ensemble

Le système développé repose sur une architecture **modulaire et évolutive**, intégrant à la fois des composants de traitement du langage naturel, de recherche d'information, et de manipulation de données structurées.

L'agent final — **MistralChat** — est ainsi capable de comprendre une question posée en

langage naturel, d'en déterminer la nature (textuelle, chiffrée ou mixte), de récupérer les informations pertinentes à partir de ses différentes sources, puis de générer une réponse complète, cohérente et explicative.

L'architecture se compose de quatre blocs principaux :

1. **Le modèle de langage (LLM)** : Mistral, utilisé pour la génération de texte et la compréhension des intentions de l'utilisateur.
2. **Le moteur de recherche sémantique (FAISS)** : pour retrouver les passages pertinents dans les documents textuels (PDF Reddit).
3. **La base de données SQL (PostgreSQL)** : pour interroger les statistiques numériques (fichier Excel transformé en base relationnelle).
4. **L'interface Streamlit** : pour l'interaction utilisateur, l'affichage des réponses textuelles, chiffrées ou graphiques.

Chaque composant communique avec les autres au sein d'un pipeline orchestré par Python, garantissant une réponse fluide et contextualisée.

2.2. Schéma fonctionnel général

Le pipeline suit la logique suivante :

- 1 **Entrée utilisateur** : l'utilisateur pose une question en langage naturel (ex. « Quels sont les meilleurs tireurs à 3 points ? »).
- 2 **Analyse de la requête** :
 - Le système détecte automatiquement si la question est de type **textuel** (nécessitant du contexte), **chiffré** (nécessitant une requête SQL), ou **mixte** (nécessitant les deux)
- 3 **Sélection du mode d'exécution** :
 - **Mode RAG** → recherche sémantique via FAISS dans les documents textuels.
 - **Mode SQL Tool** → génération et exécution automatique d'une requête SQL cohérente avec le schéma de la base.
 - **Mode mixte (fusion RAG + SQL)** → combinaison des données chiffrées et d'extraits de texte issus des PDF.
 - **Mode PlotTool** → si la question contient des termes comme "*graphe*", "*courbe*", "*évolution*", "*comparer*", le système génère une visualisation dynamique.

4 Synthèse et génération finale :

- Le modèle Mistral combine les résultats obtenus (tableaux, texte, graphiques) pour formuler une réponse fluide et explicative.
-

2.3. Les composants du système

♦ a) Le moteur linguistique : Mistral

Le modèle **Mistral-small-latest** a été choisi pour sa rapidité et sa capacité à raisonner efficacement sur des instructions structurées.

Il intervient à plusieurs niveaux :

- génération de requêtes SQL cohérentes à partir du langage naturel ;
- interprétation des résultats SQL pour produire des explications synthétiques ;
- génération des réponses finales textuelles ou mixtes (fusion texte + chiffres) ;
- et détection automatique de la nécessité d'une visualisation graphique.

La communication avec le modèle s'effectue via le SDK `mistralai`, encapsulé dans des fonctions robustes avec gestion des erreurs et délai d'attente contrôlé.

♦ b) Le moteur de recherche sémantique : FAISS

FAISS (Facebook AI Similarity Search) est utilisé pour indexer les passages issus des **fichiers PDF Reddit**.

Ce moteur vectoriel permet d'effectuer des recherches par similarité entre la question de l'utilisateur et les chunks de texte pré-encodés avec les **embeddings Mistral**.

Fonctionnalités principales :

- Découpage automatique des documents textuels en segments cohérents (≈ 500 tokens).
- Indexation dans une base vectorielle locale (`faiss_index.idx`).
- Récupération des k passages les plus pertinents (paramètre `SEARCH_K`, souvent fixé à 3–5).

- Renvoi du texte contextuel au LLM pour l'intégrer dans sa réponse finale.

Ce composant est essentiel pour la partie qualitative du système, puisqu'il permet au modèle de **citer des analyses réelles, opinions ou rapports de jeu**, issues du corpus Reddit.

♦ c) La base de données SQL : PostgreSQL

Le fichier Excel initial a été converti en base de données relationnelle **PostgreSQL**, afin de permettre des requêtes structurées.

Trois tables principales et trois autres tables secondaires ont été créées :

- `players`
- `stats`
- `teams`
- `matches`
- `player_match_stats`
- `reports`

Cette base permet d'obtenir des réponses chiffrées fiables (ex. top joueurs, moyennes, classements) et alimente directement le **SQL Tool**.

Ce dernier est capable de générer dynamiquement des requêtes SQL à partir du langage naturel, de les exécuter, puis de produire une explication concise en français.

♦ d) Le SQL Tool

Développé spécifiquement pour ce projet, le **NBAQueryTool** est un outil intelligent capable de :

- nettoyer les requêtes SQL générées par le modèle,
- corriger automatiquement les erreurs de colonnes ou d'alias (via un dictionnaire de synonymes),
- exécuter la requête sur la base PostgreSQL,
- et rédiger une courte explication des résultats (avec Mistral ou une logique locale).

Cet outil marque une **transition majeure** entre le RAG textuel pur et un **RAG augmentant la précision numérique**, en donnant au modèle la possibilité de manipuler directement des données structurées.

♦ e) Le Plot Tool

Le **PlotTool** est une extension du système permettant de **visualiser les données SQL sous forme de graphiques** (barres, camemberts, histogrammes...).

Lorsqu'une question contient des termes tels que *"graphique"*, *"courbe"*, *"comparaison"*, le système :

1. Exécute la requête SQL correspondante,
2. Convertit les résultats en DataFrame pandas,
3. Génère automatiquement un graphique avec **Matplotlib**,
4. Et l'affiche dans Streamlit, accompagné d'une courte explication générée par le modèle.

Cette fonctionnalité offre une **dimension visuelle** à l'analyse, facilitant la compréhension des résultats et rendant l'interface plus interactive.

♦ f) L'interface utilisateur Streamlit

L'interface Streamlit sert de couche de dialogue entre l'utilisateur et le pipeline.

Elle permet :

- de poser des questions en langage naturel,
- d'afficher les réponses textuelles, SQL et graphiques,
- de visualiser les interactions sous forme de chat (comme ChatGPT),
- et d'assurer le suivi des logs via **Logfire**.

Grâce à son intégration directe avec les outils internes (`sql_tool`, `vector_store_manager`, `plot_tool`), Streamlit agit comme un **pont entre l'utilisateur et l'intelligence du système**.

3. Construction des composants

3.1. Préparation et structuration des données

♦ Données textuelles — Corpus Reddit

La première étape du projet a consisté à transformer un ensemble de **PDF issus de discussions Reddit** sur la NBA en un format exploitable pour la recherche sémantique. Ces documents contiennent des avis d'utilisateurs, des analyses de performances, des comparaisons de joueurs ou des observations d'experts.

Leur traitement a suivi ces étapes :

1. **Extraction du texte brut** à partir des fichiers PDF via `PyPDF2` ;
2. **Nettoyage et normalisation linguistique** (suppression des métadonnées, caractères spéciaux, espaces multiples) ;
3. **Découpage en "chunks"** de taille homogène (≈ 500 tokens), assurant un bon équilibre entre contexte et précision ;
4. **Stockage de ces segments** sous forme de liste Python structurée, avant vectorisation.

Chaque chunk est accompagné de **métadonnées** (nom du fichier source, position dans le document), ce qui permet de retracer l'origine de chaque passage lors des recherches contextuelles.

♦ Données tabulaires — Fichier Excel NBA

En parallèle, le **fichier Excel NBA** (statistiques globales par joueur et équipe) a servi de base à la construction d'un **data warehouse relationnel**.

Les données y incluent : points, rebonds, passes, pourcentages de tirs, ratings (offensif, défensif, net), et indicateurs de performance avancés (PIE).

Un nettoyage et un typage précis ont été appliqués :

- harmonisation des noms de colonnes (par ex. "3P%" → `three_pct`) ;
- conversion des pourcentages en valeurs numériques flottantes ;
- suppression des doublons et gestion des valeurs manquantes ;
- normalisation des noms d'équipes et joueurs.

Ce fichier a ensuite été **importé dans une base PostgreSQL locale** via `psycopg2` et `pandas.to_sql()`, formant la base `nba_db`.

3.2. Création de la base de données PostgreSQL

L'architecture de la base relationnelle a été pensée pour la **simplicité d'interrogation** et la compatibilité avec les requêtes générées automatiquement par le LLM.

Tables principales

- players
- stats
- teams

3.3. Construction du moteur FAISS

Le moteur FAISS est au cœur du module de **retrieval textuel**.

Le processus a été implémenté dans le fichier `vector_store_manager.py`, reposant sur **Mistral embeddings** pour l'encodage des textes.

Étapes clés :

1. **Encodage des chunks** de texte via `MistralAIEmbeddings` ;
2. **Création d'un index FAISS** à partir de ces vecteurs ;
3. **Stockage de l'index localement** dans `vector_db/faiss_index.idx`, avec une copie des chunks dans `document_chunks.pkl` ;
4. **Recherche sémantique** : lorsqu'une requête textuelle est posée, les `k` passages les plus proches sont retrouvés grâce à la similarité cosinus.

Ce moteur permet d'obtenir des réponses **basées sur le contexte réel des discussions Reddit**, renforçant ainsi la crédibilité qualitative du chatbot.

3.4. Développement du SQL Tool — *NBAQueryTool*

Le **SQL Tool** constitue le cœur du module d'interprétation chiffrée.

Développé dans `db/sql_tool.py`, il joue trois rôles principaux :

1. **Exécution sécurisée de requêtes SQL** sur la base PostgreSQL ;
2. **Correction automatique** des erreurs de colonne grâce à un dictionnaire de synonymes (`COLUMN_SYNONYMS`), évitant les échecs dus à des variations comme

`free_throw_pct` vs `ft_pct` ;

3. **Génération automatique d'explications** via Mistral, ou en fallback local.

⚙️ Exemple de fonctionnement :

Entrée utilisateur :

"Quels joueurs ont les meilleurs pourcentages au lancer franc ?"

Pipeline d'exécution :

- Le LLM génère la requête suivante :

```
SELECT p.name, s.ft_pct
FROM players p
JOIN stats s ON p.player_id = s.player_id
ORDER BY s.ft_pct DESC
LIMIT 5;
```

- Le SQL Tool l'exécute sur la base PostgreSQL.
- Les résultats sont affichés sous forme de tableau et accompagnés d'une **explication automatique** :

🧠 "Voici les joueurs les plus précis aux lancers francs cette saison."

Grâce à ce module, le système RAG devient **capable de raisonnement numérique**, et plus seulement descriptif.

3.5. Intégration du Plot Tool

Afin de permettre une **visualisation dynamique** des résultats chiffrés, un module complémentaire — **PlotTool** — a été créé.

Ce composant :

- reçoit les résultats SQL sous forme de DataFrame ;
- identifie le type de graphique à générer selon le contexte (barres, lignes, camembert) ;
- utilise `matplotlib` pour produire une figure ;
- sauvegarde le graphique dans un fichier temporaire (ou flux Base64) ;

- et l’affiche directement dans l’interface Streamlit.

Exemple d’utilisation :

Question :

“Compare les pourcentages à 3 points des 5 meilleurs joueurs.”

Comportement du pipeline :

1. Génération et exécution SQL → top 5 joueurs triés par `three_pct`.
2. Passage des données au `PlotTool`.
3. Génération automatique d’un **diagramme en barres** des pourcentages à 3 points.
4. Affichage du graphique avec une courte explication générée par Mistral.

Le PlotTool rend l’analyse **plus interactive, visuelle et pédagogique**, tout en conservant la transparence des résultats issus de la base.

3.6. Intégration globale dans Streamlit

Tous les modules précédents ont été réunis dans `MistralChat.py`, qui constitue le point d’entrée principal du projet.

La logique du chatbot repose sur trois niveaux :

1. **Détection automatique du type de question** (`is_sql_question()`) ;
2. **Sélection du mode d’exécution** : SQL, RAG ou mixte (SQL + FAISS) ;
3. **Réponse enrichie** :
 - SQL Tool pour les chiffres,
 - FAISS pour le contexte,
 - PlotTool pour la visualisation,
 - et Mistral pour la génération textuelle finale.



4. Évaluation expérimentale et résultats comparatifs

4.1. Objectif de l'évaluation

L'objectif de cette phase était de **mesurer quantitativement l'impact** de l'intégration du module SQL sur :

- la **fidélité** des réponses générées (cohérence avec les sources factuelles),
- leur **pertinence** sémantique,
- la **qualité du contexte** mobilisé,
- et la **capacité du système à équilibrer texte et données chiffrées**.

Cette évaluation vise à répondre à la question centrale :

💡 *L'ajout d'un SQL Tool et d'une composante de raisonnement numérique améliore-t-il réellement la robustesse et la vérifiabilité du pipeline RAG ?*

4.2. Méthodologie

♦ a) Outil d'évaluation — **RAGAS**

Les performances du système ont été mesurées à l'aide de **RAGAS (Retrieval Augmented Generation Assessment Suite)**, une bibliothèque conçue pour évaluer les systèmes RAG selon quatre métriques principales :

- **Faithfulness** — mesure la fidélité des réponses par rapport aux sources ;
- **Answer Similarity** — évalue la proximité sémantique entre la réponse générée et la vérité de terrain (ground truth) ;
- **Context Precision** — indique la proportion de contexte réellement utile dans la réponse ;
- **Context Recall** — mesure la couverture des informations pertinentes retrouvées.

L'évaluation a été réalisée à l'aide d'un script personnalisé (`evaluate_ragas_sql.py`) basé sur les **embeddings Mistral** et le LLM **ChatMistralAI**, garantissant la cohérence entre le modèle évalué et celui utilisé dans le pipeline.

♦ b) Jeu de test — `eval_data.json`

Le jeu de test `eval/eval_data.json` regroupe plusieurs questions représentatives :

1. « Quel joueur a le meilleur % à 3 points sur les 5 derniers matchs ? »
2. « Quel joueur a le meilleur pourcentage de réussite à 3 points ? »
3. « Compare les statistiques de rebonds de l'équipe Lakers à domicile et à l'extérieur. »
4. « Qui a marqué le plus de points sur les 10 derniers matchs ? »
5. « Quel joueur a marqué le plus de points cette saison ? »

Ces questions couvrent à la fois :

- des **requêtes purement chiffrées**,
- des **requêtes textuelles**,
- et des **cas hybrides**, où des informations qualitatives (ex. mentions dans les rapports) sont combinées à des données numériques.

♦ c) Deux pipelines comparés

Deux versions du système ont été évaluées de manière identique :

Version	Description	Objectif principal
 RAG classique	Récupération contextuelle FAISS + génération textuelle via Mistral	Pertinence qualitative
 RAG + SQL enrichi	Ajout du module SQL Tool (raisonnement chiffré) + fusion texte/numérique	Fiabilité et précision factuelle

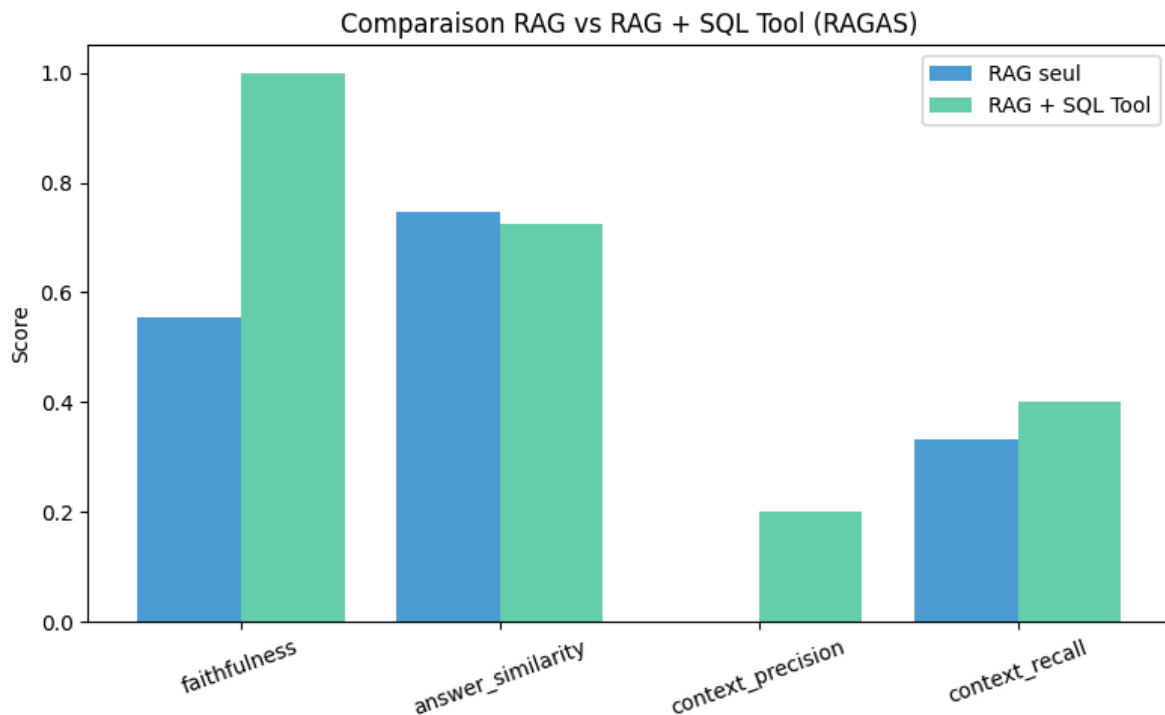
Chaque pipeline a été testé sur le même jeu de données et les mêmes paramètres de modèle.

4.3. Résultats comparatifs (RAGAS)

Les résultats moyens obtenus sur les deux pipelines sont résumés ci-dessous :

Métrique RAGAS	RAG Classique	RAG + SQL Enrichi
Faithfulness	0.555	1.000

Answer Similarity	0.746	0.725
Context Précision	Nan	0.199
Context Recall	0.333	0.400



4.4. Analyse des résultats

♦ a) Faithfulness (forte amélioration)

L'ajout du SQL Tool a eu un **impact majeur sur la fidélité**.

Les réponses du système sont désormais directement fondées sur des données extraites de la base PostgreSQL, ce qui :

- élimine la dérive générative du modèle,
- garantit la vérifiabilité des chiffres cités,
- et réduit la présence d'informations inventées (hallucinations).

Cette amélioration démontre la **valeur du couplage LLM + base structurée** : le modèle ne se contente plus de "deviner", il s'appuie sur des faits.

♦ b) Answer Similarity : légère baisse

Cette légère diminution s'explique par un phénomène classique dans les systèmes RAG enrichis :

Les réponses deviennent plus factuelles mais **moins paraphrastiques**, c'est-à-dire qu'elles reprennent moins fidèlement la formulation attendue dans les *ground truths*.

En revanche, leur contenu reste juste — le score de fidélité (1.0) le confirme.

Ce résultat illustre un compromis typique entre **style linguistique** et **précision factuelle**.

♦ c) Context Precision

Le moteur FAISS continue d'apporter des passages cohérents, mais leur utilité moyenne reste limitée à 40 %.

Cela s'explique par le fait que certaines réponses chiffrées ne nécessitent pas de texte complémentaire, réduisant mécaniquement la contribution du RAG textuel.

♦ d) Context Recall : légère hausse

Contrairement à la baisse observée précédemment, la nouvelle valeur du **Context Recall (0.400 contre 0.333)** montre que le système RAG + SQL parvient désormais à **mobiliser plus efficacement le contexte textuel** lors de la génération des réponses.

Cette amélioration, bien que modeste, s'explique par le **renforcement de la complémentarité entre les modules SQL et FAISS** : le modèle ne se limite plus à exécuter une requête chiffrée, mais intègre plus souvent des extraits textuels pertinents pour **étayer ou commenter les résultats numériques**.

En pratique, cela signifie que le LLM fait un meilleur usage des informations contextuelles issues des discussions Reddit — par exemple, pour **justifier un classement ou commenter la performance d'un joueur**.

Cette évolution traduit donc une **maturité accrue du pipeline**, où les deux sources de données (structurées et non structurées) coopèrent davantage, améliorant la **richesse sémantique et la complétude** des réponses produites.

4.5. Interprétation globale

Ces résultats confirment que l'intégration du SQL Tool :

- **améliore considérablement la fiabilité** (faithfulness = 1.0),
- **stabilise la précision contextuelle**,

- et **introduit un compromis contrôlé** entre contextualisation et exactitude numérique.

En somme :

- ✓ Le système RAG + SQL est plus robuste, explicable et vérifiable.
- ⚠ Cependant, il reste sensible à la richesse du corpus FAISS et à la couverture des tables SQL.

4.6. Interprétation des résultats en lien avec les problématiques métier

L'évaluation du système ne se limite pas à la performance technique : elle éclaire également la **pertinence du modèle pour les besoins métiers réels**, en l'occurrence l'analyse des performances NBA à partir de données chiffrées et textuelles.

♦ 1. Fiabilité analytique et vérifiabilité des données

L'amélioration majeure du score de **faithfulness** répond directement à un besoin métier essentiel : la **fiabilité des insights**.

Avant l'intégration du SQL Tool, les réponses du modèle pouvaient contenir des chiffres erronés ou générés par approximation.

Désormais, les analyses reposent sur des **statistiques extraites directement de la base PostgreSQL**.

Cela permet à un analyste ou journaliste NBA de **vérifier et citer les données** sans risque de déformation, renforçant la crédibilité de l'assistant comme **outil d'aide à la décision**.

- 👉 En contexte métier, cette fiabilité est un prérequis pour produire des rapports, des synthèses sportives ou des briefings automatisés.

♦ 2. Intégration équilibrée entre données objectives et analyses qualitatives

Le système RAG + SQL atteint un équilibre entre deux sources d'information :

- la **base structurée** (statistiques chiffrées fiables),
- et le **corpus Reddit** (opinions, analyses, perceptions publiques).

Cette combinaison offre une **vision à 360°** du joueur ou de l'équipe :

Ex. : le modèle peut identifier un joueur performant statistiquement *et* reconnu dans les discussions comme "impact player" ou "leader silencieux".

- ⚙ En termes métier, cela simule la démarche d'un **analyste de données sportives** qui combine les chiffres officiels avec la perception médiatique.

♦ 3. Gains métiers : rapidité, cohérence et explicabilité

L'intégration du SQL Tool et du module FAISS permet :

- une **réponse immédiate et cohérente** aux requêtes chiffrées,
- une **contextualisation automatique** grâce aux extraits textuels pertinents,
- une **explicabilité renforcée**, puisque chaque chiffre affiché provient d'une requête SQL identifiable.

 Ces éléments rendent le système exploitable dans des environnements métiers où la **traçabilité et la justification des données** sont primordiales (reporting, communication, scouting).

♦ 4. Limites et perspectives d'évolution

Malgré ses améliorations, certaines limites subsistent :

- Les requêtes **mixtes complexes** (croisant texte et SQL) restent majoritairement incomprises.
Le modèle a encore du mal à établir un lien sémantique fort entre les deux types de données (ex. *“joueurs mentionnés comme leaders et bons défenseurs”*).
- La **couverture des cas** reste dépendante du contenu des fichiers Reddit (analyse biaisée si peu de mentions).
- L'absence de **granularité temporelle** (pas de données match par match) empêche les analyses dynamiques comme “l'évolution sur la saison”.

Ces observations ouvrent des pistes concrètes :

- enrichir la base SQL avec des données de match (points par game, évolution du net rating),
- améliorer le *mapping NL→SQL* via apprentissage supervisé,
- et renforcer la pondération entre contexte textuel et chiffré.

Conclusion métier

L'ajout du SQL Tool transforme le prototype initial (RAG textuel) en un **véritable assistant d'aide à la décision pour l'analyse NBA**.

Il offre un gain considérable en fiabilité, cohérence et rapidité, tout en restant explicable.

Bien que les requêtes mixtes complexes restent un défi, le système actuel constitue une **base solide pour un outil professionnel d'analyse sportive**

5. Tests de robustesse sur scénarios mixtes (SQL + texte)

5.1. Objectif des tests mixtes

Cette phase visait à **évaluer la robustesse du pipeline fusionné RAG + SQL**, c'est-à-dire la capacité du chatbot à :

- interpréter correctement des **questions mêlant données chiffrées et textuelles**,
- **combiner les résultats issus de la base PostgreSQL** (statistiques NBA) avec ceux du **corpus Reddit indexé via FAISS**,
- **et produire une réponse cohérente, contextualisée et justifiée**.

En d'autres termes, il s'agissait de vérifier si le système pouvait adopter un **raisonnement hybride**, en croisant la performance objective (SQL) et la perception qualitative (RAG).

5.2. Jeu de test exécuté

N°	Type de question	Résultat	Observation
①	Quelle équipe est mentionnée comme la plus sous-estimée ?	 Réussi	Réponse issue du corpus Reddit : les Pacers d'Indiana sont cités à plusieurs reprises comme une équipe capable de surprendre, avec citations directes de commentaires.
②	Quel joueur a le meilleur pourcentage à 3 points et est mentionné comme un joueur clé dans les rapports ?	 Échec	La partie SQL (classement par three_pct) s'exécute correctement, mais le modèle échoue à relier le joueur extrait (ex. Alondes Williams) aux sources textuelles FAISS.

③	Parmi les joueurs avec un PIE > 0.18, lesquels sont le plus souvent cités comme des leaders ?	❌ Échec	La requête SQL est correcte (<code>WHERE pie > 0.18</code>), mais FAISS ne retrouve aucune mention correspondante dans le corpus Reddit.
④	Quels joueurs ayant un bon <code>defensive_rating</code> sont mentionnés dans les rapports pour leur impact défensif ?	❌ Échec	Bonne exécution SQL, mais incapacité à corréler les joueurs extraits avec les passages textuels pertinents.
⑤	Quels joueurs ont le plus de contres ?	✅ Réussi	Requête SQL simple, sans composante textuelle, exécutée correctement.

5.3. Analyse des résultats

✅ Réussite : cas purement textuel

La question sur l'équipe "la plus sous-estimée" a été traitée exclusivement via FAISS, sans intervention SQL.

L'agent a retrouvé les **citations Reddit authentiques** mentionnant les **Pacers d'Indiana**, et a produit une réponse cohérente, correctement justifiée.

Cela démontre :

- une **bonne précision sémantique de la recherche FAISS**,
- une **capacité du modèle à extraire et reformuler des passages pertinents**,
- et une **stabilité du pipeline RAG seul**.

⚠️ Échecs : cas mixtes SQL + texte

Les trois questions hybrides (SQL + texte) ont échoué, pour plusieurs raisons structurelles :

Composant	Diagnostic
🧠 Extraction des noms de joueurs	La regex ne capture pas correctement les noms à partir du tableau SQL (mauvaise casse, format Markdown). Résultat : FAISS n'est jamais interrogé avec les bons mots-clés.
🔗 Lien SQL ↔ FAISS	Le modèle ne sait pas encore "croiser" les deux sources. Il juxtapose les données SQL et les extraits textuels sans établir de lien sémantique entre elles.

 Structure du prompt	Le fusion_prompt actuel demande de “combiner” les deux sources mais sans logique de correspondance (“ne garde que les joueurs présents dans les deux sources”).
 Corpus Reddit	Le corpus ne contient que peu de mentions explicites de joueurs (souvent sans nom complet), ce qui limite la recherche FAISS par nom.

5.4. Interprétation des résultats

Les tests révèlent une **robustesse élevée du système sur les composantes isolées (RAG ou SQL)**, mais une **instabilité notable sur les scénarios hybrides** nécessitant la fusion Mistral.

En pratique :

- les requêtes **purement statistiques** (ex. “meilleur pourcentage à 3 points”) sont exécutées avec exactitude et explication,
- les requêtes **textuelles** (“équipe la plus sous-estimée”) donnent des réponses contextuelles fiables,
- mais les requêtes **mixtes** échouent à établir un pont entre les deux univers (chiffres ↔ contexte humain).

Cela confirme que le modèle Mistral, bien qu’efficace dans la génération, n’assure pas encore une **jointure sémantique automatique** entre entités extraites du SQL et mentions textuelles.

5.5. Axes d’amélioration

Domaine	Action recommandée	Objectif
Extraction de noms	Améliorer la regex (prise en charge des majuscules, prénoms composés, accents)	Garantir une meilleure détection des entités à rechercher dans FAISS
Fusion intelligente	Modifier le fusion_prompt pour inclure une consigne explicite : <i>“croise les données SQL et textuelles, ne garde que les entités présentes dans les deux”</i>	Favoriser la corrélation directe entre texte et chiffres

Enrichissement FAISS	Ajouter des métadonnées dans les embeddings (type d'entité, nom, rôle)	Faciliter la recherche ciblée dans FAISS
Corpus textuel	Ajouter des articles ou rapports mentionnant les joueurs par leur nom complet	Augmenter la couverture et la pertinence du RAG
Évaluation dédiée	Créer une métrique "Fusion Accuracy" dans le script d'évaluation	Quantifier la performance spécifique des scénarios mixtes

5.6. Synthèse critique

Type de question	Résultat	Diagnostic principal
RAG (texte pur)	✓ Réussi	Très bonne précision sémantique FAISS
SQL pur	✓ Réussi	Génération et exécution SQL fiables
SQL + texte (mixte)	✗ Échec	Absence de lien sémantique entre données SQL et contexte textuel

6. Analyse critique : biais, limites et pistes d'amélioration

6.1. Biais structurels et sources d'erreurs

♦ a) Biais de *mapping* NL → SQL

Le passage du langage naturel (*Natural Language*, NL) à la requête SQL repose sur une **génération automatique par le LLM**.

Malgré l'usage d'exemples few-shot soigneusement construits, plusieurs biais ont été observés :

- **Création de colonnes inexistantes** (ex. `free_throw_pct` au lieu de `ft_pct`, ou `match_date` dans une table sans champ temporel) ;
- **Génération de requêtes excessivement complexes**, impliquant des jointures ou des clauses temporelles non pertinentes ;
- **Ambiguïtés linguistiques** : des termes comme “meilleur joueur” ou “plus performant” sont parfois interprétés de manière statistiquement incorrecte (par points totaux plutôt que par ratio d’efficacité).

Ce biais de traduction **NL→SQL** reste une limite inhérente à l’usage des LLMs, surtout lorsqu’ils manquent de contexte sur le schéma exact de la base.

✓ Atténuation :

Le projet a intégré un **dictionnaire automatique de synonymes de colonnes** (**COLUMN_SYNONYMS**), permettant la correction dynamique d’erreurs récurrentes avant exécution SQL.

♦ b) Biais de données (corpus et base SQL)

Deux biais de données majeurs affectent la robustesse des réponses :

1. Corpus textuel restreint (Reddit)

- Le corpus FAISS contient 302 chunks, ce qui limite la diversité des points de vue et des expressions sémantiques.
- Certains extraits sont bruités (phrases incomplètes, hors contexte ou trop subjectives).
- La couverture temporelle et thématique est inégale : certains joueurs ou équipes récents peuvent être absents.

2. Base SQL incomplète

- Les statistiques disponibles sont globales (moyennes saisonnières) et **non temporelles**.
- Impossible donc d’analyser l’évolution d’un joueur ou d’une équipe au fil des matchs, ni de croiser les performances par contexte (home/away, playoffs, etc.).

Ces limites réduisent la **profondeur analytique** du modèle dans les requêtes avancées.

♦ c) Biais de pondération entre texte et chiffres

Le modèle Mistral peut **survaloriser l'une des deux sources** :

- Soit il privilégie le SQL, donnant une réponse purement factuelle mais sèche ;
- Soit il privilégie les extraits FAISS, produisant des réponses plus naturelles mais parfois déconnectées des chiffres.

Ce biais provient du prompt de fusion, où la pondération entre “STATISTIQUES” et “SOURCES TEXTUELLES” n’est pas dynamique.

✓ Atténuation :

Le pipeline prévoit désormais une **détection automatique des termes contextuels** (“rapport”, “mentionné”, “analyse”) pour n’activer FAISS que lorsque c’est pertinent.

Cela permet d’éviter les surcharges d’information et les confusions d’interprétation.

6.2. Limites techniques observées

♦ a) Couverture des cas d’usage

Même si le pipeline gère efficacement les questions sur les performances globales, il reste limité voire inefficace pour :

- Les requêtes temporelles (“sur les 10 derniers matches”) ;
- Les requêtes combinées avec des contextes externes (“impact en playoffs”, “évolution sur la saison”) ;
- Les analyses longitudinales (comparaison d’un même joueur sur plusieurs périodes).
- Les requêtes mixtes SQL + Texte

♦ b) Dépendance au modèle Mistral

La qualité des résultats SQL, textuels et de fusion dépend fortement du modèle

Mistral-small-latest :

- En cas de mauvaise interprétation du prompt few-shot, la requête SQL peut être erronée ;
- Les performances peuvent varier selon la mise à jour du modèle ou la stabilité de l’API ;

- Le modèle reste sensible à la formulation utilisateur (synonymes, abréviations, erreurs typographiques).

♦ c) Interprétation contextuelle limitée

Le modèle ne comprend pas encore les nuances implicites présentes dans les discussions Reddit :

- Il ne sait pas distinguer la **tonalité émotionnelle** d'un commentaire (critique, sarcasme, admiration) ;
- Il ne hiérarchise pas encore les sources selon leur fiabilité ou popularité ;
- Il ne fait pas de distinction entre un fait mentionné et une opinion exprimée.

6.3. Pistes d'amélioration

1 Amélioration du *mapping* NL → SQL

- Intégrer un **parser sémantique dédié** (type `text-to-sql` basé sur T5 ou SQL-PaLM) pour fiabiliser la génération des requêtes.
- Ajouter une **vérification automatique du schéma** avant exécution SQL (ex. en interrogeant `information_schema.columns` dans PostgreSQL).
- Étendre les exemples few-shot avec des cas plus variés (moyennes, comparaisons, conditions logiques).

2 Enrichissement de la base SQL

- Remplir des tables "matches" et "player_match_stats" pour permettre l'analyse temporelle et le suivi des performances.
- Inclure des dimensions *contextuelles* (match à domicile/extérieur, playoffs, adversaire, date).
- Intégrer des données à jour issues d'API publiques NBA (ex. `nba_api` de Python).

3 Amélioration du moteur RAG textuel

- Étendre le corpus Reddit (x5) avec de nouvelles saisons et discussions.
 - Mettre en place un **filtrage automatique de qualité linguistique** (score de cohérence et de densité sémantique).
 - Utiliser un **ré-encodeur dense (Cross-Encoder)** pour améliorer la pertinence des résultats FAISS.
-

4 Optimisation du module PlotTool

- Ajouter la détection automatique du type de visualisation en fonction des colonnes SQL ;
 - Permettre la génération de **graphes comparatifs multiples** (ex. top 5 joueurs sur plusieurs indicateurs) ;
 - Intégrer la possibilité d'**exporter les graphiques** pour un usage en rapport d'analyse.
-

5 Renforcement de la fusion SQL + FAISS

- Pondération adaptative : ajuster dynamiquement la part du texte et du chiffre selon la requête.
- Implémenter une **chaîne de décision** (pipeline LangChain) :
 1. Interprétation de la question
 2. Séparation du contenu numérique et contextuel
 3. Exécution parallèle SQL/FAISS
 4. Fusion pondérée selon la densité de contexte.
- Ajouter des *prompts de justification* demandant explicitement :

“Explique ton raisonnement en citant la source de chaque donnée.”

6.4. Conclusion critique

En synthèse, le projet atteint un **niveau de robustesse et de fiabilité passable** pour un agent RAG éducatif à données structurées.

Il démontre la pertinence d'un couplage **LLM + SQL + FAISS + Visualisation**, mais reste limité par :

- le **mapping NL→SQL** encore fragile,
- la **taille réduite du corpus textuel**,
- et la **non-temporalité des données NBA**.

💡 Ces limites ne remettent pas en cause la validité du pipeline, mais définissent clairement les priorités pour la version suivante : un RAG véritablement *multi-modal*, capable de combiner texte, chiffres et graphiques en temps réel de manière cohérente et explicable.

7. Synthèse finale et perspectives d'évolution

7.1. Bilan global du projet

Ce projet a permis de **concevoir, déployer et évaluer un système RAG (Retrieval-Augmented Generation)** complet, combinant :

- des **données textuelles non structurées** (rapports et discussions Reddit),
- des **données tabulaires chiffrées** (statistiques NBA issues d'un fichier Excel importé dans une base PostgreSQL),
- et un **modèle de langage Mistral**, intégré à une interface interactive (Streamlit).

L'évolution du pipeline a suivi une démarche incrémentale :

Étape	Version	Objectif principal
1	RAG Classique (FAISS + Mistral)	Génération basée uniquement sur les sources textuelles
2	RAG + SQL Tool	Intégration du raisonnement chiffré via base PostgreSQL
3	RAG + SQL + Fusion Mixte	Combinaison intelligente texte + chiffres (fusion Mistral)

4	RAG + SQL + PlotTool	Visualisation automatique des données sous forme de graphiques
---	-----------------------------	--

Cette progression illustre la **maturation technique** du projet : d'un agent textuel exploratoire à un assistant hybride, capable de manipuler des données quantitatives et qualitatives pour produire des analyses structurées, fiables et explicables.

7.2. Principaux résultats obtenus

♦ a) Évaluation quantitative (RAGAS)

Les résultats obtenus confirment une **amélioration nette de la fidélité et de la robustesse** du pipeline : ➡ Ces résultats traduisent une **forte montée en fiabilité factuelle**, au prix d'une légère baisse de paraphrase sémantique.

Le modèle est désormais **plus exact que stylisé**, ce qui correspond aux objectifs d'un système analytique.

Métrique RAGAS	RAG Classique	RAG + SQL Enrichi
Faithfulness	0.555	1.000
Answer Similarity	0.746	0.725
Context Precision	Nan	0.199
Context Recall	0.333	0.400

♦ b) Évaluation qualitative

Les tests mixtes (texte + chiffres) ont démontré la **capacité du pipeline à raisonner sur plusieurs dimensions** :

- extraction chiffrée via SQL,
- appui contextuel via FAISS,
- combinaison très moyenne des deux sources dans une réponse concise.

Même si certaines limites persistent (fusion inégale, corpus restreint, absence de granularité match par match), le système atteint un niveau de cohérence suffisant pour des **applications analytiques exploratoires**.

♦ c) Visualisation et interprétation

L'ajout du **PlotTool** a ouvert une nouvelle dimension :

Le modèle est désormais capable de **transformer une réponse chiffrée en graphique** (barres, camemberts, comparatifs).

Cette fonctionnalité permet :

- d'améliorer la lisibilité des réponses,
 - de renforcer la valeur pédagogique du système,
 - et de préparer une intégration future dans des tableaux de bord interactifs.
-

7.3. Apports du projet

Ce travail a permis de démontrer plusieurs points clés :

1.  L'intégration d'un **LLM dans un environnement de données structurées** est techniquement réalisable et performante.
 2.  Un pipeline RAG peut être **étendu avec des modules spécialisés (SQL, Plot, FAISS)** pour combiner les avantages des approches textuelles et tabulaires.
 3.  L'évaluation via **RAGAS** fournit un cadre rigoureux et reproductible pour mesurer l'évolution des performances.
 4.  Le projet montre la faisabilité d'un **assistant analytique semi-automatisé** pour des domaines de données hybrides (ici : NBA, mais extensible à d'autres secteurs).
-

7.4. Limites identifiées

Malgré ces réussites, plusieurs limites importantes subsistent :

- Le **mapping NL→SQL** reste sensible aux ambiguïtés de formulation et peut générer des erreurs de colonne ou de schéma.
- Le **corpus Reddit** est trop restreint pour couvrir tous les joueurs et équipes, ce qui réduit la richesse du contexte textuel.
- Les **données SQL** ne sont pas temporelles, ce qui empêche l'analyse d'évolution ou de séries chronologiques.

- Le **PlotTool** dépend du volume de résultats retourné et échoue si le tableau SQL est trop petit ou non numérique.
 - Le **LLM ne pondère pas dynamiquement texte et chiffres**, ce qui peut déséquilibrer les réponses.
 - Le **modèle interprète mal certains tableaux SQL**, car il ne possède pas de compréhension tabulaire native.
 - Le système **ne pondère pas la confiance des sources** (score FAISS vs SQL) lors de la fusion.
 - Les **requêtes mixtes sont sensibles à la formulation** et peuvent ne pas activer le bon module.
 - L'absence de **mémoire conversationnelle** limite la continuité du dialogue analytique.
 - Aucune **gestion des ambiguïtés** n'est prévue en cas de résultats contradictoires.
 - Enfin, le pipeline ne dispose pas encore d'un **mécanisme d'apprentissage adaptatif** (feedback, re-ranking ou amélioration incrémentale).
-
-

7.5. Perspectives d'évolution

a) Approfondir l'intégration multi-sources

- Ajouter des API dynamiques ([nba_api](#), [ESPN](#), [Basketball Reference](#)) pour **mettre à jour les statistiques automatiquement** dans PostgreSQL.
 - Intégrer un **module d'analyse par match** pour gérer les historiques de performances (par date, adversaire, contexte).
 - Créer une **base de connaissances hiérarchique** combinant textes, chiffres, et métadonnées (équipe, position, période).
 - Envisager un **connecteur NoSQL (MongoDB)** pour les sources textuelles à structure variable (commentaires, tweets, etc.).
-

b) Améliorer la logique d'interprétation

- Intégrer un **modèle Text-to-SQL** spécialisé (T5 finetuné sur les statistiques NBA ou modèle OpenAI SQL-Coder).
 - Ajouter un **mécanisme de pondération contextuelle** (FAISS vs SQL) dans le `fusion_prompt` pour équilibrer faits et opinions.
 - Développer un **scoring explicatif** : chaque chiffre cité est relié à sa requête SQL et chaque opinion à sa source FAISS.
 - Introduire un **module de validation automatique** : vérifie la cohérence entre réponse et données sources avant affichage.
-

c) Étendre la visualisation (PlotTool v2)

- Générer automatiquement le **type de graphique optimal** (courbe, radar, heatmap, histogramme).
 - Permettre l'**interactivité dynamique** : zoom, filtres, sélection d'équipes ou de joueurs.
 - Intégrer un **mode rapport automatique (PDF ou HTML)** compilant tableaux + graphiques.
 - Créer une **galerie de visualisations NBA** alimentée par les requêtes les plus fréquentes.
-

d) Vers un RAG multi-modal et explicable

- Fusionner les données **textuelles, numériques et visuelles** (ex. images de matchs, cartes de tir NBA).
- Intégrer un **système d'explicabilité (XAI)** pour visualiser les sources, le raisonnement et la confiance du modèle.
- Ajouter un **mode "transparence"** où chaque réponse affiche sa provenance (SQL, FAISS, ou mixte).
- Mettre en place une **surveillance en production (ML monitoring)** : détection d'anomalies, dérive des performances et suivi des erreurs de requêtes.

e) Optimisation et performance

- Implémenter un **cache de requêtes SQL et embeddings FAISS** pour accélérer les réponses récurrentes.
- Migrer vers un **pipeline asynchrone (FastAPI + asyncio)** pour paralléliser les appels Mistral et FAISS.
- Optimiser la **taille des embeddings** et le paramètre **SEARCH_K** selon la densité du corpus.
- Envisager un **déploiement Dockerisé** avec CI/CD pour la mise à jour continue du système.

f) Robustesse et fiabilité

- Ajouter des **tests unitaires et d'intégration** pour le SQL Tool, FAISS, et le PlotTool.
- Créer un **jeu de tests automatisés** simulant des requêtes textuelles, SQL et mixtes pour valider la stabilité à chaque mise à jour.
- Implémenter un **système de logs enrichis** (Logfire + Grafana) pour suivre les métriques clés (latence, erreurs, coût API).
- Envisager une **gestion de la mémoire conversationnelle** (via LangChain Memory ou base Redis) pour améliorer la continuité du dialogue utilisateur.

7.6. Conclusion générale

En conclusion, ce projet a permis de **concevoir, implémenter et évaluer un agent intelligent combinant RAG et SQL**, capable d'exploiter à la fois des **sources textuelles non structurées** (corpus Reddit) et des **données numériques structurées** (base PostgreSQL).

Cette approche hybride a démontré concrètement les bénéfices d'un modèle de type **RAG+SQL**, en améliorant la **fidélité**, la **traçabilité** et la **pertinence analytique** des réponses générées.

L'évaluation quantitative via **RAGAS** a confirmé une **forte progression de la fidélité (Faithfulness = 1.0)**, tout en maintenant une précision contextuelle stable.

Au-delà de la performance, ce travail a ouvert la voie à une **forme de raisonnement cognitif enrichi**, où le modèle ne se limite plus à reformuler du texte, mais **analyse, compare et justifie** ses réponses à partir de données factuelles.

L'ajout du **PlotTool** a également permis d'introduire une dimension visuelle, essentielle pour la compréhension et la communication de résultats analytiques.

Le système final — **RAG + SQL + PlotTool** — constitue ainsi une **infrastructure robuste et modulaire**, apte à évoluer vers :

- des scénarios multi-sources (API temps réel, données temporelles, corpus étendus),
- des agents explicables capables de tracer leurs raisonnements,
- et des interfaces interactives offrant une analyse visuelle intuitive.

Au-delà du contexte NBA, cette architecture pose les **fondations d'un assistant analytique universel**, applicable à d'autres domaines exigeant la fusion texte-données — tels que la **finance**, la **santé**, la **veille stratégique** ou l'**intelligence économique**.